

Lecture 6 — Polymorphism and Operator Overloading

- Goal: Understand how the SAME action can behave DIFFERENTLY depending on the object.
- Part 1: Polymorphism (method overriding, duck typing).
- Part 2: Operator Overloading (custom +, ==, etc.).
- We will follow a step-by-step style like Lecture 5, and end with practice problems.

Learning Objectives

- Explain polymorphism and why it matters in OOP.
- Write classes that override the same method with different behavior.
- Use one function to work with different object types.
- Understand how operator overloading works in Python using special methods (`__add__`, `__mul__`, `__eq__`, `__str__`).
- Apply these ideas to business-style classes instead of math examples.

What is Polymorphism?

- Poly = many, Morph = forms → 'many forms'.
- Idea: The same function name is used, but each object decides what it does.
- Example: All employees have `calculate_pay()`, but each role calculates pay differently.
- Why it's useful: You write general code once, and let each class customize behavior.

Polymorphism vs Inheritance

- Inheritance: Child class gets structure/behavior from parent (is-a).
- Polymorphism: Child class REDEFINES behavior of the same method name.
- Without polymorphism, you would write separate function names like `pay_for_manager()`, `pay_for_intern()`, etc. → messy.
- With polymorphism, all of them just implement `calculate_pay()`.

Example Setup — Base Class Employee

- Employee is a general template.
- name and base_salary are common to all employees.
- calculate_pay() gives a default payment rule.
- We will now create specialized employee types that change this rule.

```
class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary

    def calculate_pay(self):
        # default rule
        return self.base_salary
```

Subclass: Manager (Override Method)

- Manager inherits from Employee.
- We call `super().__init__` to set shared data.
- Manager ADDS a new field: `bonus`.
- Manager REDEFINES `calculate_pay()`.

```
class Manager(Employee):  
    def __init__(self, name, base_salary, bonus):  
        super().__init__(name, base_salary)  
        self.bonus = bonus  
  
    def calculate_pay(self):  
        # Manager salary = base + bonus  
        return self.base_salary + self.bonus
```

Subclass: SalesPerson (Override Method)

- SalesPerson also inherits Employee.
- Adds commission (money earned from sales).
- Also overrides calculate_pay().
- Note: both Manager and SalesPerson have calculate_pay() but different logic.

```
class SalesPerson(Employee):  
    def __init__(self, name, base_salary, commission):  
        super().__init__(name, base_salary)  
        self.commission = commission  
  
    def calculate_pay(self):  
        # SalesPerson salary = base + commission  
        return self.base_salary + self.commission
```

Using Polymorphism

- We call the SAME method name `calculate_pay()` on different objects.
- Each object runs its OWN version of the method.
- This is runtime polymorphism: Python decides which version to run based on the object.

```
m = Manager("Aimen", 70000, 15000)
s = SalesPerson("Sara", 40000, 8000)

print(m.name, "Pay:", m.calculate_pay())
print(s.name, "Pay:", s.calculate_pay())
```

Why Polymorphism is Powerful

- Your code calls one method name, like `calculate_pay()`.
- Every subclass promises to implement that method its own way.
- So your business logic becomes easier to extend → Add Intern, Contractor, etc. without changing old code.
- This is extremely common in payroll, billing, tax rules, etc.

Duck Typing in Python

- Python does not force classes to be related by inheritance.
- Rule: 'If it looks like a duck and quacks like a duck, it's a duck.'
- Meaning: If two unrelated objects both have the same method name, Python will just call it.
- So in Python, polymorphism does NOT always require inheritance.

Duck Typing Example

- PDFReport and ExcelReport are NOT in inheritance relationship.
- Both just happen to have .generate().
- run_report() works with anything that has generate().
- That is duck typing. This is Python-style polymorphism.

```
class PDFReport:
    def generate(self):
        print("Generating PDF report...")

class ExcelReport:
    def generate(self):
        print("Generating Excel spreadsheet...")

def run_report(report_obj):
    # we don't care what the exact class is,
    # we just expect it to have generate()
    report_obj.generate()

pdf = PDFReport()
xls = ExcelReport()
run_report(pdf)
run_report(xls)
```

Operator Overloading — Motivation

- We already do $10 + 5$ and also `'Hi' + '!'` — same operator (+), different meanings.
- In Python, classes can define what `+`, `-`, `*`, `==`, `<`, `str()` etc. should do for THEIR objects.
- This is called operator overloading.
- We do this by writing special methods like `__add__`, `__mul__`, `__eq__`, `__str__`, etc.

Business Example Class: PriceTag

- We'll model prices for products or services.
- Each PriceTag will store an amount and a currency code (like 'PKR' or 'USD').
- We will overload:
 - `__add__` → combine two prices (same currency),
 - `__mul__` → multiply cost by quantity,
 - `__str__` → nice string printing like 'PKR 1500.00'.
- This is more realistic than math vector examples.

```
class PriceTag:
    def __init__(self, amount, currency):
        self.amount = amount      # number
        self.currency = currency  # string like "PKR"

    def __str__(self):
        # called when you print(obj)
        return f"{self.currency} {self.amount:.2f}"
```

Overloading `__add__` (Price + Price)

- `__add__` is automatically called when you use `+`.
- We check if both prices are in the same currency.
- We return a NEW PriceTag with the combined amount.
- So now `+` works on our custom class.

```
class PriceTag:
    def __init__(self, amount, currency):
        self.amount = amount
        self.currency = currency

    def __str__(self):
        return f"{self.currency} {self.amount:.2f}"

    def __add__(self, other):
        # Add two PriceTag objects
        if self.currency != other.currency:
            raise ValueError("Currency mismatch")
        total_amount = self.amount + other.amount
        return PriceTag(total_amount, self.currency)

p1 = PriceTag(1500, "PKR")
p2 = PriceTag(2000, "PKR")
p3 = p1 + p2          # calls p1.__add__(p2)
print(p3)             # PKR 3500.00
```

Overloading `__mul__` (Price × Quantity)

- `__mul__` is called when you do `object * number`.
- We compute total cost for 'qty' units.
- We again return a NEW PriceTag object.
- Now our class supports math-like usage in billing.

```
class PriceTag:
    def __init__(self, amount, currency):
        self.amount = amount
        self.currency = currency

    def __str__(self):
        return f"{self.currency} {self.amount:.2f}"

    def __add__(self, other):
        if self.currency != other.currency:
            raise ValueError("Currency mismatch")
        return PriceTag(self.amount + other.amount, self.currency)

    def __mul__(self, qty):
        # multiply price by quantity to compute total cost
        total = self.amount * qty
        return PriceTag(total, self.currency)

item_price = PriceTag(499.99, "PKR")
total_bill = item_price * 3      # calls __mul__
print(total_bill)               # PKR 1499.97
```

Improving Readability with `__str__`

- `__str__(self)` controls how your object prints.
- Without `__str__`, `print(obj)` shows something ugly like
- With `__str__`, you get business-friendly output like 'PKR 1499.97'.
- Good for reports, invoices, receipts.

Key Ideas So Far

- Polymorphism: Same method name, different behavior for different objects.
- Method Overriding: Child class gives a new version of a parent method.
- Duck Typing: Polymorphism in Python without inheritance; 'if it has the method, we trust it'.
- Operator Overloading: Redefine how +, *, ==, etc. work for YOUR objects using magic methods like `__add__`, `__mul__`, `__str__`.
- All of this improves clarity and reusability in real systems.

Practice Problem 1 — Payroll System

- Create classes: Employee (name, base_salary), Intern (name, base_salary, stipend), Contractor (name, hourly_rate, hours).
- Each class should have a method monthly_pay() with its own logic.
- Task: Create one object of each type and print their monthly_pay().
- Question: Where did you use polymorphism? Where did you use overriding?

Practice Problem 2 — Shopping Cart with Operator Overloading

- Create a class ItemPrice with amount and currency.
- Implement __str__, __add__, __mul__ exactly like PriceTag from lecture.
- Make two items in the same currency, add them to get cart_total.
- Multiply one item price by a quantity to get line_total.
- Print both totals in readable form.

Practice Problem 3 — Report Generator (Duck Typing)

- Create two unrelated classes: `HtmlReport` and `TextReport`.
- Both must define `generate()` but in different styles.
- Write a function `run(report_obj)` that just calls `report_obj.generate()`.
- Create one `HtmlReport` and one `TextReport` and pass both into `run()`.
- Question: Did you need inheritance here? Why or why not?

Summary / Takeaways

- Polymorphism lets different objects answer the same 'question' in their own way.
- Operator overloading lets your objects behave naturally with +, *, etc.
- These tools make code easier to scale, test, and read in business applications.
- Next lecture will apply all of this in a full system design case study.